

RFM is one of the most common customer segmentation techniques used in marketing and data analytics.

RFM = Recency + Frequency + Monetary

Customer	Last Purchase	No. of Orders	Total Spent
A	5 days ago	15	₹35,000
B	120 days ago	3	₹4,000
C	20 days ago	8	₹12,000

1. Recency (R)

How recently did the customer purchase?

Formula:

Recency = Analysis Date - Last Purchase Date

Example:

- Analysis date = 1 Aug 2026

Customer A:

- Last purchase = 27 Jul 2026

Recency = 1 Aug - 27 Jul
= 5 days

Lower Recency is **better** because the customer bought recently.

2. Frequency (F)

How many times has the customer purchased?

Formula:

Frequency = Number of unique orders

Example:

Customer A

Order ID

101

102

103

104

Frequency = 4

Higher Frequency is **better**.

3. Monetary (M)

Question: *How much money has the customer spent?*

Formula:

Monetary = Sum of Amount

Example

Order	Amount
--------------	---------------

101	₹500
-----	------

102	₹900
-----	------

103	₹700
-----	------

104	₹400
-----	------

Monetary = 500 + 900 + 700 + 400
= ₹2500

Higher Monetary is **better**.

Sample Dataset:

Customer	Invoice	Date	Amount
A	101	1 Jan	500
A	102	15 Jan	900
A	103	28 Jan	700
B	104	2 Jan	1000
B	105	5 Feb	500
C	106	25 Jan	2000

Assuming today's date as **1 Feb**.

Customer A

Last Purchase = 28 Jan

Recency

1 Feb - 28 Jan = 4 days

Frequency

3 orders

Monetary

$500+900+700 = 2100$

Result

$R = 4$

$F = 3$

$M = 2100$

Customer B

Recency

1 Feb - 15 Jan

import pandas as pd

```
df = pd.DataFrame({
    "CustomerID": [1,1,1,2,2,3],
    "InvoiceNo": [101,102,103,104,105,106],
    "InvoiceDate": [
        "2024-01-01",
        "2024-01-15",
        "2024-01-28",
        "2024-01-02",
        "2024-02-05",
        "2024-01-25"
    ],
    "Amount": [500,900,700,1000,500,2000]
})
```

```
df["InvoiceDate"] = pd.to_datetime(df["InvoiceDate"])
```

Choose an analysis date:

```
snapshot_date = df["InvoiceDate"].max() + pd.Timedelta(days=1)
```

Calculate RFM:

```
rfm = df.groupby("CustomerID").agg( {
    "InvoiceDate": lambda x: (snapshot_date - x.max()).days,
    "InvoiceNo": "count",
    "Amount": "sum"
})
```

```
rfm.columns = ["Recency", "Frequency", "Monetary"]
```

Output:

CustomerID	Recency	Frequency	Monetary
1	9	3	2100
2	1	2	1500
3	12	1	2000

Recency (days)	R Score
0–10	5
11–20	4
21–30	3
31–60	2
>60	1

Frequency	F Score
1	1
2	2
3–4	3
5–7	4
8+	5

Then you combine them:

R = 5

F = 4

M = 5

Monetary (₹ Spent)	M Score
₹0–₹1,000	1
₹1,001–₹5,000	2
₹5,001–₹10,000	3
₹10,001–₹20,000	4
>₹20,000	5

(Higher is better because the customer spends more.)

For Customer A has:

- **Recency = 15 days** → **R = 5**
- **Frequency = 7 orders** → **F = 4**
- **Monetary = ₹18,000** → **M = 4**

RFM Score = 544.

Use **quintiles (20% groups)** :

For example, if you have **1,000 customers**:

- Top 20% spenders → **M = 5**
- Next 20% → **M = 4**
- Next 20% → **M = 3**
- Next 20% → **M = 2**
- Bottom 20% → **M = 1**

The same idea is used for **Frequency**, while **Recency is reversed** (most recent customers get the highest score).

This percentile-based approach is what you'll most often encounter in data analytics and in interview.

For today():as snapshot date

```
import pandas as pd
```

```
snapshot_date = pd.Timestamp.today().normalize()
```

or

```
from datetime import datetime
```

```
snapshot_date = datetime.today()
```

calculate Recency:

```
rfm["Recency"] = (snapshot_date - rfm["LastPurchaseDate"]).dt.days
```

sWhy do many tutorials use:

```
snapshot_date = df["InvoiceDate"].max() + pd.Timedelta(days=1)
```